Q9: Python Code Generation Using Structured Prompting

Aim: To generate Python code using a structured prompt and verify its correctness by executing test cases.

Algorithm

1. Start the program.
2. Define a structured prompt containing the task, input, output, and constraints.
3. Generate Python code for calculating the factorial of a number.
4. Execute the generated code.
5. Test the code using sample inputs.
6. Display the output and validation results.
7. Stop the program.

PROGRAM:

```
# Q9: Python Code Generation Using Structured Prompting

# Step 1: Define a structured prompt
prompt = """
Task: Write a Python program to calculate factorial.
Input: A non-negative integer.
Output: Factorial of the given number.
Constraint: Negative numbers must raise ValueError.
"""

print("Structured Prompt:")
print(prompt)

# Step 2: Python code generated for the task
def factorial(n):
    if n < 0:
        raise ValueError("Number must be non-negative")

    result = 1
    for i in range(1, n + 1):
        result *= i

    return result

# Step 3: Test the code
print("\nTest Results:")
print("Factorial of 0:", factorial(0))
print("Factorial of 3:", factorial(3))
print("Factorial of 5:", factorial(5))

# Step 4: Validate the results
assert factorial(0) == 1
assert factorial(3) == 6
assert factorial(5) == 120

try:
    factorial(-2)
except ValueError:
    print("Negative input test: Passed")

print("All test cases passed!")
```

OUTPUT:

```
Structured Prompt:

Task: Write a Python program to calculate factorial.
Input: A non-negative integer.
Output: Factorial of the given number.
Constraint: Negative numbers must raise ValueError.


Test Results:
Factorial of 0: 1
Factorial of 3: 6
Factorial of 5: 120
Negative input test: Passed
All test cases passed!
```

Result

Thus, structured prompting was demonstrated by defining clear requirements, implementing the corresponding Python code, and validating its output with test cases.